Amazon Cognito

SDK Authentication Flows, User Pool / Identity Pool Architecture,
and the SRP Cryptographic Protocol — Technical Reference

This reference covers how a web or mobile client integrates with Amazon Cognito end to end: the two SDK layers available, every user pool authentication flow (SRP, plain password, custom Lambda-driven, and the newer choice-based flow), multi-factor authentication, password reset, token refresh and validation, and how a Cognito Identity Pool exchanges user pool tokens for temporary AWS credentials. A dedicated section walks through the Secure Remote Password (SRP) protocol's cryptography in detail, since it is the default and recommended sign-in mechanism.

1. Two Ways to Talk to Cognito

Every Cognito integration sits on one of two layers, and it's worth being explicit about which one a given code sample uses, because the ergonomics are very different.

High-level: Amplify Auth (Gen 2)

Handles token storage, silent refresh, and the SRP cryptographic math internally. The client calls friendly functions — signUp(), signIn(), confirmSignIn(), fetchAuthSession() — and inspects a nextStep field to figure out what the user needs to do next in a multi-step flow.

Low-level: the cognito-idp API directly

InitiateAuth, RespondToAuthChallenge, and (for trusted backends) AdminInitiateAuth. This is what Amplify calls under the hood. Using it directly means managing challenge state, session tokens, and — if not using a library like amazon-cognito-identity-js — the SRP math yourself.

This document shows both layers side by side wherever it's useful, since seeing the low-level shape makes it much clearer what the high-level SDK is actually doing for you.

2. Sign-Up Flow

A new user is created in the user pool directory in an UNCONFIRMED state until they verify a code sent to their email or phone.

Amplify (JavaScript / TypeScript)

```
import { signUp } from 'aws-amplify/auth';

const { isSignUpComplete, userId, nextStep } = await signUp({
  username: 'jane@example.com',
  password: 'Str0ngP@ssw0rd!',
  options: {
    userAttributes: {
      email: 'jane@example.com',
      phone_number: '+15551234567',
    },
    autoSignIn: true, // auto sign-in once confirmation completes
  },
});
// nextStep.signUpStep is typically "CONFIRM_SIGN_UP"
```

Low-level (@aws-sdk/client-cognito-identity-provider)

```
import { CognitoIdentityProviderClient, SignUpCommand }
  from '@aws-sdk/client-cognito-identity-provider';

const client = new CognitoIdentityProviderClient({ region: 'us-east-1' });

await client.send(new SignUpCommand({
  ClientId: APP_CLIENT_ID,
  Username: 'jane@example.com',
  Password: 'Str0ngP@ssw0rd!',
  UserAttributes: [
    { Name: 'email', Value: 'jane@example.com' },
    { Name: 'phone_number', Value: '+15551234567' },
  ],
  // SecretHash required only if the app client has a client secret
  // (native / public clients should NOT have one)
}));
```

If a Pre sign-up Lambda trigger is configured, it fires here — commonly used to auto-confirm trusted users or block disposable email domains.

3. Confirmation Flow

```
import { confirmSignUp, resendSignUpCode } from 'aws-amplify/auth';

await confirmSignUp({ username: 'jane@example.com', confirmationCode: '123456' });

// if the code expired or wasn't received:
await resendSignUpCode({ username: 'jane@example.com' });
```

Low-level equivalents are ConfirmSignUpCommand and ResendConfirmationCodeCommand. A successful confirmation fires the Post confirmation Lambda trigger — the common place to write a row into your own application database.

4. Sign-In Flows — Comparison

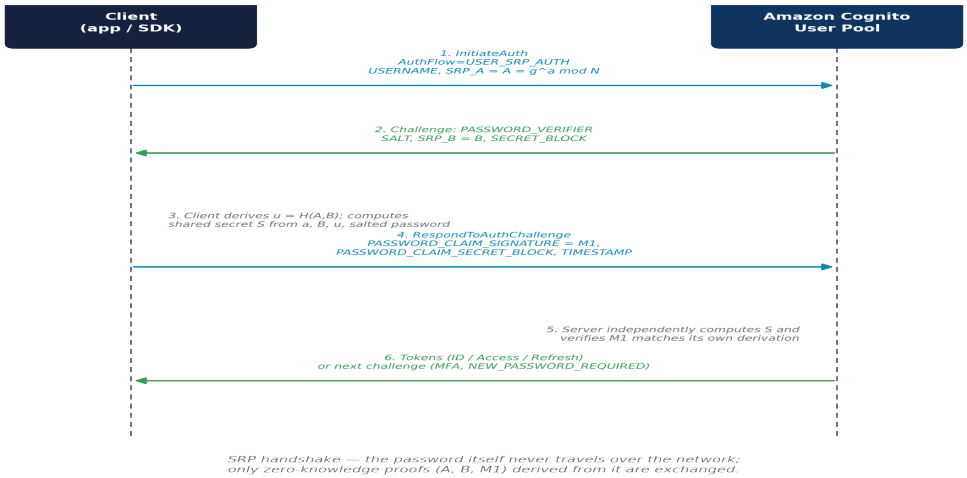
Cognito supports several distinct AuthFlow values on InitiateAuth. Picking the right one matters — this is the part of Cognito integrations most often implemented incorrectly.

AuthFlow	What it does	When to use
USER_SRP_AUTH	Secure Remote Password — the password itself never crosses the wire, even under TLS	Default, recommended for password sign-in
USER_PASSWORD_AUTH	Sends username + password directly (TLS-protected only)	Legacy clients, server-side scripts, migrations
CUSTOM_AUTH	Fully Lambda-driven challenge sequence	Passwordless (OTP, magic link, custom logic)

AuthFlow	What it does	When to use
USER_AUTH	Newer choice-based flow: PASSWORD_SRP, EMAIL_OTP, SMS_OTP, WEB_AUTHN as first-class options	New apps wanting passkeys / OTP without custom Lambda triggers
REFRESH_TOKEN_AUTH	Exchanges a refresh token for new ID / access tokens	Silent re-authentication
ADMIN_USER_PASSWORD_AUTH / ADMIN_NO_SRP_AUTH	Server-side, admin-privileged sign-in	Backend services, migration scripts (needs cognito-idp:AdminInitiateAuth)

5. USER_SRP_AUTH — The Sign-In Handshake

This is the flow behind the scenes whenever an app calls signIn() with a password and Amplify's default authFlowType. It is a genuine cryptographic protocol exchange, not just a single request/response.



The USER_SRP_AUTH challenge/response sequence between client and Cognito user pool.

Amplify (hides all the cryptography)

```
import { signIn, confirmSignIn } from 'aws-amplify/auth';

const { isSignedIn, nextStep } = await signIn({
  username: 'jane@example.com',
  password: 'Str0ngP@ssw0rd!',
  // options.authFlowType defaults to USER_SRP_AUTH
});

if (!isSignedIn) {
  switch (nextStep.signInStep) {
    case 'CONFIRM_SIGN_IN_WITH_SMS_CODE':
    case 'CONFIRM_SIGN_IN_WITH_TOTP_CODE':
    case 'CONFIRM_SIGN_IN_WITH_EMAIL_CODE': {
      await confirmSignIn({ challengeResponse: userEnteredCode });
      break;
    }
    case 'CONTINUE_SIGN_IN_WITH_MFA_SELECTION': {
      await confirmSignIn({ challengeResponse: 'SOFTWARE_TOKEN_MFA' });
      break;
    }
    case 'CONFIRM_SIGN_IN_WITH_NEW_PASSWORD_REQUIRED': {
      await confirmSignIn({ challengeResponse: newPassword });
      break;
    }
  }
}
```

Low-level, using amazon-cognito-identity-js (implements the SRP math for you)

```
import { CognitoUserPool, CognitoUser, AuthenticationDetails }
  from 'amazon-cognito-identity-js';

const pool = new CognitoUserPool({ UserPoolId: 'us-east-1_xxxxx', ClientId: APP_CLIENT_ID });
const user = new CognitoUser({ Username: 'jane@example.com', Pool: pool });
const authDetails = new AuthenticationDetails({
  Username: 'jane@example.com',
  Password: 'Str0ngP@ssw0rd!',
});

user.authenticateUser(authDetails, {
  onSuccess: (session) => {
    const idToken = session.getIdToken().getJwtToken();
    const accessToken = session.getAccessToken().getJwtToken();
    const refreshToken = session.getRefreshToken().getToken();
  },
  onFailure: (err) => console.error(err),
  mfaRequired: (challengeName, challengeParameters) => {
    const code = promptUserForCode();
    user.sendMFACode(code, { onSuccess: () => {}, onFailure: () => {} });
  },
  newPasswordRequired: (userAttributes, requiredAttributes) => {
    user.completeNewPasswordChallenge(newPassword, {}, { onSuccess: () => {} });
  },
});
```

This library performs the exact A / B / M1 math described in the next section internally — that is precisely what an “SRP client library” exists to do.

6. Secure Remote Password (SRP) — Protocol Deep Dive

SRP is a Password-Authenticated Key Exchange (PAKE) protocol, standardized in outline by RFC 2945 and the specific parameter set AWS uses following the conventions of RFC 5054. Cognito's implementation of USER_SRP_AUTH is this protocol adapted to the InitiateAuth / RespondToAuthChallenge API shape. The property that makes it worth using instead of sending a password directly: two parties can prove to each other that they share the same secret (the password) without either one ever transmitting that secret, or anything from which it could be trivially recovered, across the network — even over a connection with no encryption at all. TLS on top of this is defense in depth, not the thing actually protecting the credential.

6.1 Fixed parameters and setup

- N — a large safe prime (the modulus). Fixed and public, shared by every client and the server, chosen once for the system.
- g — a generator value, also fixed and public.
- H() — a cryptographic hash function (SHA-256 in Cognito's implementation).
- At sign-up, the server never stores the raw password. It stores only a per-user salt and a verifier v = g^x mod N, where x is derived by hashing the salt together with the username and password. The password itself, and x, are discarded after the verifier is computed.

6.2 Step-by-step handshake

1. **Client generates an ephemeral keypair.** It picks a random private value a, and computes the public ephemeral value A = g^a mod N. It sends InitiateAuth with AuthFlow=USER_SRP_AUTH, USERNAME, and SRP_A = A.
2. **Server responds with a PASSWORD_VERIFIER challenge.** It looks up the user's stored salt and verifier v, picks its own random private value b, and computes its public ephemeral value B = (k·v + g^b) mod N, where k is a fixed multiplier derived as k = H(N, g). It returns SALT, SRP_B = B, and a SECRET_BLOCK (an encrypted blob Cognito uses internally to bind the challenge to this specific session).
3. **Both sides derive the scrambling parameter** u = H(A, B). This value ties the two ephemeral public keys together and prevents an attacker who intercepts one session from reusing values in another.
4. **Client computes the shared secret S.** Using its password, the received salt, its own private value a, the server's public value B, and u: S = (B – k·g^x)^(a + u·x) mod N, where x is derived the same way it was at sign-up (from salt, username, and password). Critically, this requires the actual password — a client that doesn't know it cannot compute the correct S.

5 **Server computes the same shared secret independently**, using the stored verifier v instead of the password: S = (A·v^u)^b mod N. If both sides did the math correctly with the same password, these two derivations of S are mathematically guaranteed to be equal — this is the core algebraic identity SRP relies on.

6 **Both sides derive a session key K = H(S)**, and from it the client computes a proof value M1 = HMAC(K, username, pool_id, secret_block, timestamp). The client sends this back via RespondToAuthChallenge with ChallengeName=PASSWORD_VERIFIER, as PASSWORD_CLAIM_SIGNATURE, together with PASSWORD_CLAIM_SECRET_BLOCK and TIMESTAMP.

7 **Server verifies.** It computes its own expected M1 from the S it derived, and compares. A match proves the client possesses the correct password without the password ever having been transmitted. On success, Cognito returns the AuthenticationResult (ID, access, and refresh tokens) or, if further steps are configured, the next challenge (an MFA challenge or NEW_PASSWORD_REQUIRED).

6.3 Why this matters in practice

- **No password on the wire, ever.** Even if TLS were somehow misconfigured or intercepted, an eavesdropper only sees A, B, salt, and proof values — none of which allow recovering the password directly (recovering it would require solving a discrete-logarithm problem over N).
- **Server never stores the password, or anything as sensitive as it.** Only a salt and a verifier are persisted. A database compromise doesn't hand over crackable password hashes in the traditional sense — the verifier can't be used to authenticate without solving the same discrete-log problem.
- **Mutual authentication.** Because both sides derive S independently and prove it, the protocol also gives the client some assurance it's really talking to a server that has the correct verifier — not just the server authenticating the client.
- **Replay resistance.** Fresh random ephemeral values (a, b) each session, plus the u scrambling parameter and a timestamp in the proof, mean a captured handshake transcript can't be replayed later to authenticate again.

6.4 The full authenticated user workflow, end to end

Putting the pieces together, a typical web or mobile client goes through this sequence from first launch to calling an AWS service directly:

1 **Sign-up.** SignUp / signUp() creates the user record; status is UNCONFIRMED.

2 **Confirmation.** User enters the code from email/SMS; ConfirmSignUp / confirmSignUp() moves the account to CONFIRMED. The Post confirmation Lambda trigger may run here.

3 **Sign-in via SRP.** InitiateAuth(USER_SRP_AUTH) → PASSWORD_VERIFIER challenge → RespondToAuthChallenge with the computed proof, exactly as walked through in section 6.2.

4 **Optional MFA challenge.** If enabled, an SMS_MFA, SOFTWARE_TOKEN_MFA, or EMAIL_OTP challenge is returned instead of tokens; the client answers it before tokens are issued.

5 **Tokens issued.** The client receives an ID token, an access token, and a refresh token, all signed JWTs (except the refresh token, which is opaque).

6 **Token exchange with the Identity Pool (if AWS resource access is needed).** The ID token is handed to GetId, then GetCredentialsForIdentity on the Identity Pool, which validates it and returns short-lived AWS access key / secret key / session token via AWS STS.

7 **Direct AWS service calls.** The client uses those temporary credentials to call S3, DynamoDB, or any other IAM-permitted service directly, with no server in the middle.

8 **Silent refresh.** When the access token nears expiry, InitiateAuth(REFRESH_TOKEN_AUTH) (or Amplify's fetchAuthSession) obtains new tokens without bothering the user, as long as the refresh token is still valid.

9 **Sign-out.** A local sign-out simply discards local tokens; a global sign-out revokes every refresh token issued to that user across all devices, though already-issued access/ID tokens remain valid until their natural expiry since they are stateless JWTs.

Sections 7 through 13 below cover each of these steps (other than sign-up/confirmation, already shown above) with full code samples.

7. USER_PASSWORD_AUTH — Simple, Opt-In

The app client must explicitly allow this flow; it is disabled by default precisely because SRP is the preferred mechanism. It sends the username and password directly, relying solely on TLS for protection in transit. It remains useful for backend scripts, admin tooling, and user migrations where a full SRP client isn't practical.

```
# boto3, server-side
import boto3
client = boto3.client('cognito-idp', region_name='us-east-1')

resp = client.initiate_auth(
    ClientId=APP_CLIENT_ID,
    AuthFlow='USER_PASSWORD_AUTH',
    AuthParameters={'USERNAME': 'jane@example.com', 'PASSWORD': 'Str0ngP@ssw0rd!'},
)
if 'ChallengeName' in resp:
    print(resp['ChallengeName'], resp['ChallengeParameters'])
else:
    tokens = resp['AuthenticationResult']  # AccessToken, IdToken, RefreshToken, ExpiresIn
```

8. CUSTOM_AUTH — Lambda-Driven Passwordless

Three Lambda triggers coordinate a fully custom challenge sequence:

- **DefineAuthChallenge** — a state machine that looks at the session history (previous challenges and whether they were passed) and decides what challenge comes next, or whether to issue tokens.
- **CreateAuthChallenge** — generates the actual challenge (e.g. emails a one-time code and stores its hash for later comparison).
- **VerifyAuthChallengeResponse** — checks the user's submitted answer against what was generated.

```
// Client kicks off CUSTOM_AUTH
const { nextStep } = await signIn({
  username: 'jane@example.com',
  options: { authFlowType: 'CUSTOM_WITHOUT_SRP' }, // or customWithSRP for SRP-first custom flows
});
// nextStep.signInStep === 'CONFIRM_SIGN_IN_WITH_CUSTOM_CHALLENGE'

const result = await confirmSignIn({ challengeResponse: otpFromUser });
```

At the low level, this is InitiateAuth with AuthFlow=CUSTOM_AUTH, followed by repeated RespondToAuthChallenge calls with ChallengeName=CUSTOM_CHALLENGE, until DefineAuthChallenge returns issueTokens: true.

9. USER_AUTH — Choice-Based Flow (Passkeys, OTP)

Cognito's newer unified sign-in flow. InitiateAuth with AuthFlow=USER_AUTH returns either a specific challenge or, when multiple methods are enabled on the app client, a SELECT_CHALLENGE step listing

the available options: PASSWORD_SRP, EMAIL_OTP, SMS_OTP, and WEB_AUTHN (passkeys).

```
// Amplify: preferring passkeys where available
const { nextStep } = await signIn({
  username: 'jane@example.com',
  options: { authFlowType: 'USER_AUTH' },
});
if (nextStep.signInStep === 'CONFIRM_SIGN_IN_WITH_PASSKEY') {
  // the browser/platform WebAuthn ceremony happens via Amplify internally
}
```

10. Multi-Factor Authentication Challenges

When MFA is required, InitiateAuth / RespondToAuthChallenge returns an intermediate challenge name instead of tokens:

- **SMS_MFA** — respond with SMS_MFA_CODE
- **SOFTWARE_TOKEN_MFA** — a TOTP code from an authenticator app; respond with SOFTWARE_TOKEN_MFA_CODE
- **EMAIL_OTP** — a code emailed to the user; respond with that code
- **SELECT_MFA_TYPE** — the user picks which second factor to use
- **MFA_SETUP** — the user has no second factor registered yet and must call AssociateSoftwareToken plus VerifySoftwareToken (for TOTP) before continuing

Setting up TOTP MFA (Amplify)

```
import { setUpTOTP, verifyTOTPSetup } from 'aws-amplify/auth';

const totpSetup = await setUpTOTP();
const setupUri = totpSetup.getSetupUri('MyApp'); // render as a QR code
await verifyTOTPSetup({ code: userEnteredCode });
```

Low-level TOTP verification via challenge response (boto3)

```
resp = client.respond_to_auth_challenge(
    ClientId=APP_CLIENT_ID,
    ChallengeName='SOFTWARE_TOKEN_MFA',
    Session=session_from_previous_response,
    ChallengeResponses={'USERNAME': 'jane@example.com', 'SOFTWARE_TOKEN_MFA_CODE': '123456'},
)
```

11. Forgot Password / Reset Flow

```
import { resetPassword, confirmResetPassword } from 'aws-amplify/auth';

const { nextStep } = await resetPassword({ username: 'jane@example.com' });
// nextStep.resetPasswordStep === 'CONFIRM_RESET_PASSWORD_WITH_CODE'

await confirmResetPassword({
  username: 'jane@example.com',
  confirmationCode: '123456',
  newPassword: 'NewStr0ngP@ss!',
});
```

Low-level equivalents are ForgotPasswordCommand followed by ConfirmForgotPasswordCommand.

12. Refresh Token Flow

```
resp = client.initiate_auth(
    ClientId=APP_CLIENT_ID,
    AuthFlow='REFRESH_TOKEN_AUTH',
    AuthParameters={'REFRESH_TOKEN': stored_refresh_token},
)
new_tokens = resp['AuthenticationResult']  # new IdToken, AccessToken
                                            # (+ RefreshToken if rotation is on)
```

If refresh token rotation is enabled on the app client, every refresh invalidates the old refresh token and issues a new one — the new token must be persisted every time, or the session breaks on the next refresh. Amplify's fetchAuthSession({'{'} forceRefresh: true {'}'}) handles this rotation transparently.

13. Token Structure and Validation

A successful authentication returns three tokens with distinct purposes:

- **ID token** — claims about the user (sub, email, custom attributes). aud is your app client ID; token_use is “id”.
- **Access token** — scopes and authorization (client_id, scope). It carries no aud claim — validate client_id instead; token_use is “access”.
- **Refresh token** — opaque, used only against Cognito itself; not a JWT you decode.

Validating a token yourself (for example, inside a custom Lambda authorizer — API Gateway's built-in Cognito authorizer usually does this step for you already):

```
import jwt
from jwt import PyJWKClient

jwks_client = PyJWKClient(
    f"https://cognito-idp.{region}.amazonaws.com/{user_pool_id}/.well-known/jwks.json"
)
signing_key = jwks_client.get_signing_key_from_jwt(token)

claims = jwt.decode(
    token,
    signing_key.key,
    algorithms=["RS256"],
    audience=APP_CLIENT_ID,          # for ID tokens; skip for access tokens
    issuer=f"https://cognito-idp.{region}.amazonaws.com/{user_pool_id}",
)
assert claims["token_use"] == "id"    # or "access"
```

14. Global Sign-Out

```
import { signOut } from 'aws-amplify/auth';
await signOut({ global: true }); // revokes all refresh tokens for the user, across devices
```

The low-level GlobalSignOutCommand (which needs the access token) invalidates every refresh token issued to that user. Existing access and ID tokens remain valid until they naturally expire, since they are stateless JWTs — a common gotcha when short revocation windows matter, and a reason to keep access token TTLs short if instant revocation is a requirement.

15. Identity Pool — Exchanging Tokens for AWS Credentials

This is the second half of the overall workflow: a separate Cognito service and API, used once the app needs to call other AWS services directly rather than just knowing who the user is.

```
import { CognitoIdentityClient, GetIdCommand, GetCredentialsForIdentityCommand }
  from '@aws-sdk/client-cognito-identity';

const client = new CognitoIdentityClient({ region: 'us-east-1' });
const loginKey = `cognito-idp.us-east-1.amazonaws.com/${USER_POOL_ID}`;

const { IdentityId } = await client.send(new GetIdCommand({
  IdentityPoolId: IDENTITY_POOL_ID,
  Logins: { [loginKey]: idToken }, // the ID token from the user pool sign-in
}));

const { Credentials } = await client.send(new GetCredentialsForIdentityCommand({
  IdentityId,
  Logins: { [loginKey]: idToken },
}));
// Credentials.AccessKeyId, SecretKey, SessionToken, Expiration
```

In practice, these two calls are rarely made directly — a credential provider wraps them:

```
import { fromCognitoIdentityPool } from '@aws-sdk/credential-provider-cognito-identity';
import { S3Client, ListObjectsV2Command } from '@aws-sdk/client-s3';

const s3 = new S3Client({
  region: 'us-east-1',
  credentials: fromCognitoIdentityPool({
    clientConfig: { region: 'us-east-1' },
    identityPoolId: IDENTITY_POOL_ID,
    logins: { [loginKey]: idToken },
  }),
});
await s3.send(new ListObjectsV2Command({ Bucket: 'my-bucket' }));
```

For unauthenticated / guest access, Logins is omitted entirely: GetId returns a guest identity, and the IAM role attached to the pool's unauthenticated role governs what it can touch.

16. Mobile: Kotlin (Android) and Swift (iOS)

Both platforms go through the identical USER_SRP_AUTH protocol described in section 6; Amplify's native libraries embed the same SRP math in Kotlin and Swift respectively.

Android (Kotlin)

```
Amplify.Auth.signIn("jane@example.com", "Str0ngP@ssw0rd!",
    { result -> Log.i("Auth", "signed in: ${result.isSignedIn}") },
    { error -> Log.e("Auth", "sign in failed", error) }
)
```

iOS (Swift)

```
Amplify.Auth.signIn(username: "jane@example.com", password: "Str0ngP@ssw0rd!") { result in
    switch result {
    case .success(let signInResult):
        print("Signed in: \(signInResult.isSignedIn)")
    case .failure(let error):
        print("Sign in failed: \(error)")
    }
}
```

Practical notes worth flagging: USER_PASSWORD_AUTH and ADMIN_NO_SRP_AUTH must be explicitly enabled on the app client — both are off by default precisely because SRP is preferred. Access tokens cannot be individually revoked before their natural expiry (only refresh tokens can, via global sign-out); if instant revocation matters for your threat model, keep access token TTLs short rather than relying on revocation alone.

This document is an original technical write-up prepared to accompany Amazon Cognito's public SDK and API reference documentation. Flow names, challenge names, and API shapes (InitiateAuth, RespondToAuthChallenge, GetCredentialsForIdentity, and related actions) are AWS's published API surface, described here in original wording for reference purposes.

Primary references: AWS Documentation, “Amazon Cognito user pools authentication flow” and “Authentication flows” — docs.aws.amazon.com/cognito/latest/developerguide/ ; AWS Amplify Gen 2 Documentation — docs.amplify.aws